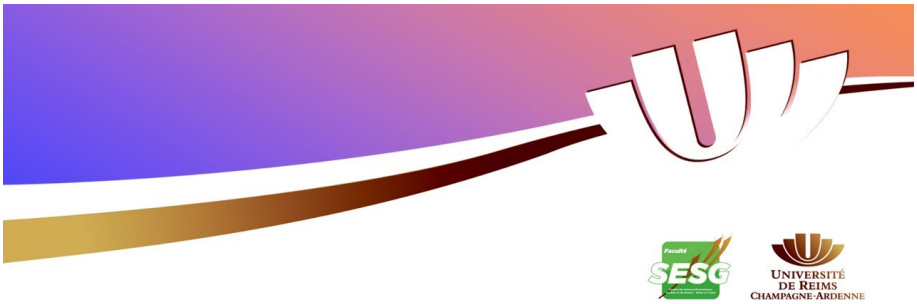


Projet d'apprentissage non supervisé

valant évaluation dans le cadre de :
Diplôme : À compléter
Année universitaire : 2026–2027
Module d'enseignement : Apprentissage non supervisé
Responsable : Frédéric BLANCHARD, Philippe REGNAULT
Comptant pour : À compléter

L'algorithme Affinity Propagation

Hervé ADJAKOSSA & Marie DE AMORIM PEREIRA DA SILVA

Métriques

Finalisé le : 2026-11-05
Page(s) : 12
Références(s) : –
Figure(s) : –
Table(s) : –
Théorème(s) : 0

Résumé : Affinity Propagation (AP) est un algorithme de classification non supervisée qui regroupe des observations en choisissant, parmi elles, des exemplaires réels, sans fixer le nombre de groupes à l'avance. Ce rapport présente ses fondements mathématiques (échanges de messages de responsabilité et de disponibilité sur une matrice de similarités), ses paramètres de réglage (préférence et amortissement), et une application reproductible au jeu de données Iris en R (apcluster) et en Python (scikit-learn). Une comparaison à K-means, K-medoids et à la classification hiérarchique, menée à nombre de groupes fixé, illustre les forces et les limites de la méthode.

Mots-clés : clustering, Affinity Propagation, exemplaires, apcluster, scikit-learn, Iris

Matériel supplémentaire :
git : https ://gitlab-mi.univ-reims.fr/dea0014/affinity_propagation
:

Table des matières

1 Introduction **3**

2 Pourquoi faire appel à Affinity Propagation ? **3**
2.1 Le problème pratique . . . 3
2.2 Ce qu'AP apporte . . . 3
2.3 Positionnement dans la zoologie des méthodes . . . 4

3 Fondements mathématiques **4**
3.1 Données : la matrice de similarités . . . 4
3.2 La préférence s(k, k) . . . 4
3.3 Fonction objectif . . . 4
3.4 Les messages : responsabilités et disponibilités . . . 5
3.5 Amortissement et convergence . . . 5
3.6 Complexité et limites . . . 5

4 Contexte d'utilisation : données, techniques, résultats attendus **6**
4.1 Quelles données ? . . . 6
4.2 Quelles techniques ? . . . 6
4.3 Quels résultats attendre ? . . . 6

5 Application au jeu de données Iris **6**
5.1 Protocole . . . 6
5.2 Résultats reproduits localement . . . 7
5.3 Comparaison qualitative avec les méthodes de référence . . . 8
5.4 Pourquoi le nombre de groupes varie-t-il ? . . . 8

6 Ce que dit la bibliographie sur Affinity Propagation **8**

7 Discussion critique **9**

8 Conclusion **9**

9 Annexe : extraits de code **9**

Bibliographie **11**

1. Introduction

Dans le cadre du module d'apprentissage non supervisé, enseigné par Frédéric Blanchard et Philippe Regnault, nous étudions l'algorithme **Affinity Propagation** (propagation d'affinité, AP dans la suite), proposé par Frey et Dueck (2007).

L'apprentissage automatique consiste à utiliser des algorithmes pour apprendre des structures, des relations ou des règles à partir de données, plutôt que de programmer manuellement toutes les règles. On distingue classiquement l'apprentissage **supervisé**, qui apprend une relation entre des données et des résultats connus afin de prédire ces résultats sur de nouvelles observations, de l'apprentissage **non supervisé**, qui travaille sur des données sans étiquettes et cherche à en révéler l'organisation. Ce projet relève de la seconde famille, et plus précisément du *clustering* : répartir des observations en groupes homogènes, appelés clusters, sans connaître ces groupes à l'avance.

Un algorithme est un processus systématique de résolution d'un problème ; un algorithme d'apprentissage automatique utilise les données pour construire la solution. Il existe de nombreuses méthodes de clustering parce que les problèmes ne se ressemblent pas : partitionnement (K-means), hiérarchie (regroupement ascendant), densité (DBSCAN), et propagation de messages, qui est la voie d'AP.

Dans ce rapport, nous expliquons le fonctionnement d'AP, ses fondements mathématiques, ses paramètres et ses limites, puis nous l'appliquons au jeu de données Iris en R (package `apcluster`) et en Python (`scikit-learn`), avec une comparaison à K-means, K-medoids et à la classification hiérarchique. Le code complet et reproductible est fourni dans le dépôt Git (`tp/`).

2. Pourquoi faire appel à Affinity Propagation ?

2.1. Le problème pratique

Dans de nombreuses situations réelles, on souhaite regrouper des objets — profils de patients, séquences génétiques, conditions expérimentales, documents — **sans savoir combien de groupes** il convient de former, et en souhaitant que chaque groupe soit représenté par un **objet réel** plutôt que par une moyenne abstraite.

K-means, la méthode de référence, impose deux choix gênants : fixer le nombre de groupes k avant l'analyse, et représenter chaque groupe par son centroïde, un point moyen qui n'existe pas dans les données. Quand le but est de sélectionner des cas typiques à examiner de près — un prototype à fabriquer, un document à lire, une condition expérimentale à reproduire —, un représentant fictif ne convient pas.

2.2. Ce qu'AP apporte

AP répond à ces deux limites : il choisit comme représentants des **exemplaires** (*exemplars*) pris **parmi les observations elles-mêmes**, et il **ne demande pas de fixer le nombre de clusters à l'avance** (Frey et Dueck 2007). Ce nombre émerge de l'algorithme, piloté par un paramètre de **préférence** qui exprime, pour chaque observation, sa propension à devenir exemplaire.

Cas d'usage documentés dans la littérature : regroupement de gènes par profil d'expression (Leone, Sumedha et Weigt 2007), sélection de conditions expérimentales représentatives en combustion (Y. Wang et al. 2024), amélioration du réglage d'AP par méta-heuristiques (Sinha et Jana 2025) et suivi incrémental de clusters dans des corpus diachroniques (Periti et al. 2025). Un carnet de démonstration en Python, qui reprend l'exemple officiel de scikit-learn, a également guidé

notre mise en œuvre (Sarkar 2019).

2.3. Positionnement dans la zoologie des méthodes

Le clustering regroupe plusieurs familles :

- les **méthodes de partitionnement** (K-means, K-medoids), qui répartissent directement les observations en un nombre de groupes fixé à l'avance ;
- les **méthodes hiérarchiques**, qui construisent une arborescence de groupes emboîtés, représentée par un dendrogramme que l'on découpe ensuite ;
- les **méthodes fondées sur la densité** (DBSCAN), qui cherchent des zones de forte concentration et acceptent du bruit.

AP n'entre dans aucune de ces cases : c'est une méthode de **propagation de messages** sur un graphe dont les nœuds sont les observations. Elle partage avec K-medoids l'idée de représentants réels, mais s'en distingue par le mécanisme (échanges itératifs de messages plutôt que recherche directe) et par le réglage du nombre de clusters (préférence continue plutôt que k entier).

3. Fondements mathématiques

3.1. Données : la matrice de similarités

Soient $x_1, \dots, x_n$ les observations. AP ne travaille pas sur les observations elles-mêmes mais sur une **matrice de similarités** S de terme général $s(i,k)$, où $s(i,k) > s(i,k')$ signifie que x_k ressemble davantage à x_i que $x_{k'}$. La similarité n'a pas besoin d'être symétrique ni de vérifier l'inégalité triangulaire : ce n'est pas nécessairement une distance.

Lorsque l'objectif est de minimiser l'erreur quadratique, Frey et Dueck (2007) utilisent le négatif de la distance euclidienne au carré :

$$s(i,k) = -\|x_i - x_k\|^2, \qquad i \neq k.$$

C'est le choix que nous adoptons, disponible dans `apcluster` sous le nom `negDistMat(r = 2)` et équivalent à l'option `affinity="euclidean"` de scikit-learn.

3.2. La préférence s(k, k)

Les valeurs **diagonales** $s(k,k)$ jouent un rôle à part : ce sont les **préférences**, qui quantifient *a priori* la propension de chaque observation à être choisie comme exemplaire. Le cas d'usage le plus courant attribue la même valeur p à tous les points ; ce paramètre unique **contrôle le nombre de clusters** : une préférence proche du minimum des similarités conduit à peu de groupes, une préférence proche du maximum en produit beaucoup. Frey et Dueck (2007) recommandent la **médiane** des similarités comme point de départ, réglage par défaut du package `apcluster` (Bodenhofer, Kothmeier et Hochreiter 2011). C'est ce paramètre, et non un nombre de groupes k, qui remplace le réglage habituel du clustering.

3.3. Fonction objectif

AP recherche une affectation $c : \{1, \dots, n\} \to \{1, \dots, n\}$, où $c(i)$ est l'exemplaire de l'observation i, qui maximise la similarité totale

$$\mathcal{S}(c) = \sum_{i=1}^{n} s\big(i, c(i)\big)$$

sous des **contraintes de cohérence** : chaque observation est affectée à exactement un exemplaire, et toute observation choisie comme exemplaire par au moins une autre doit être son propre exemplaire ($\exists i : c(i) = k \Rightarrow c(k) = k$). Trouver le maximum exact de ce critère est un problème d'optimisation combinatoire difficile — le problème de la k-médiane en est un cas particulier ; AP en cherche une bonne solution approchée par passage de messages (Frey et Dueck 2007).

3.4. Les messages : responsabilités et disponibilités

L'algorithme fait s'échanger entre les paires d'observations deux types de messages, qui peuvent être lus comme des tableaux de log-vraisemblances :

— la **responsabilité** $r(i,k)$, envoyée de l'observation i au candidat exemplaire k, mesure à quel point x_k est un exemplaire approprié pour x_i, **compte tenu des autres candidats** ;

— la **disponibilité** $a(i,k)$, envoyée du candidat k à l'observation i, mesure à quel point il serait approprié que x_i choisisse x_k, **compte tenu du soutien que x_k reçoit des autres observations**.

Les deux matrices sont initialisées à zéro, puis mises à jour par alternance :

$$r(i,k) \;\leftarrow\; s(i,k) - \max_{k' \neq k} \big\{ a(i,k') + s(i,k') \big\},$$

$$a(i,k) \;\leftarrow\; \min\Big\{ 0,\; r(k,k) + \sum_{i' \notin \{i,k\}} \max\big(0, r(i',k)\big) \Big\} \quad (i \neq k), \qquad a(k,k) \;\leftarrow\; \sum_{i' \neq k} \max\big(0, r(i',k)\big).$$

À tout moment, l'observation k est un exemplaire si son **autosupport** est positif : $r(k,k) + a(k,k) > 0$. Chaque observation est ensuite affectée à l'exemplaire k qui maximise $a(i,k) + r(i,k)$. Ces règles s'interprètent comme une instance de l'algorithme **max-sum** sur un graphe de facteurs, qui minimise une approximation de l'énergie libre (Frey et Dueck 2007).

3.5. Amortissement et convergence

Sur certains jeux de données, les mises à jour peuvent **osciller** sans jamais se stabiliser (par exemple quand des similarités sont symétriques et les préférences égales, ce qui crée plusieurs points fixes). Pour l'éviter, chaque message est **amorti** : sa nouvelle valeur vaut λ fois sa valeur précédente plus $(1-\lambda)$ fois sa valeur calculée, avec $\lambda \in [0,1)$, typiquement $\lambda = 0{,}5$ à $0{,}9$ (Frey et Dueck 2007). Ce sont bien les **messages** qui sont amortis, et non les préférences. Un λ plus élevé améliore la stabilité au prix d'un plus grand nombre d'itérations. L'algorithme s'arrête lorsque les exemplaires restent inchangés pendant un nombre d'itérations fixé (`convits`), ou au bout d'un nombre maximal d'itérations (`maxits`).

3.6. Complexité et limites

Avec une matrice de similarités dense, la mémoire requise est en $O(n^2)$ et chaque itération coûte $O(n^2)$; le coût total est donc de l'ordre de $O(n^2 T)$, où T est le nombre d'itérations jusqu'à convergence. AP convient donc surtout aux jeux de données **petits à moyens** ; pour les grands jeux, il faut passer par des similarités creuses. Le choix de la préférence reste l'une des principales

difficultés pratiques de la méthode (Sinha et Jana 2025), et une heuristique classique du problème p-médiane peut obtenir une erreur plus faible qu'AP à temps de calcul comparable (Brusco et Köhn 2008) : la supériorité d'AP n'est donc pas automatique et doit être vérifiée empiriquement, cas par cas.

4. Contexte d'utilisation : données, techniques, résultats attendus

4.1. Quelles données ?

AP s'applique dès lors que l'on sait définir une similarité entre paires d'observations : vecteurs numériques (mesures physiques, profils d'expression génétique), mais aussi données non vectorielles dès qu'une similarité existe (séquences, images). Les prétraitements usuels s'imposent : standardisation des variables pour qu'aucune ne domine la distance, choix réfléchi de la similarité.

4.2. Quelles techniques ?

En R, le package **apcluster** (Bodenhofer, Kothmeier et Hochreiter 2011) fournit `apcluster()` (matrice de similarités ou fonction de similarité, préférence par quantile `q`, amortissement `lam`, `maxits`, `convits`), ainsi que `apclusterK()` lorsqu'un nombre de groupes précis est visé. En Python, la classe `sklearn.cluster.AffinityPropagation` de scikit-learn (Pedregosa et al. 2011) offre les paramètres correspondants (`preference`, `damping`, `max_iter`, `convergence_iter`). Les deux implémentations partagent la même définition de la similarité euclidienne négative au carré.

4.3. Quels résultats attendre ?

Le résultat d'AP est une liste d'**exemplaires** — des observations réelles — et l'affectation de chaque observation à l'un d'eux. Le nombre de groupes obtenu **ne doit jamais être présenté comme « optimal »** : il dépend de la préférence et de la matrice de similarités choisies. Une étude de sensibilité (balayage de la préférence) et une comparaison à des méthodes de référence sont donc indispensables avant toute interprétation.

5. Application au jeu de données Iris

5.1. Protocole

Nous illustrons AP sur le jeu **Iris** (150 fleurs, 4 mesures : longueur et largeur des sépales et des pétales, 3 espèces de 50 individus), disponible dans R (`datasets::iris`) et dans scikit-learn. Ce jeu, facile, sert à **valider la chaîne de traitement** ; il ne démontre rien sur la performance générale d'AP. Protocole, identique en R et en Python :

1. standardisation des 4 variables (z-score) ;
2. similarité : moins la distance euclidienne au carré, $s(i,k) = -\|x_i - x_k\|^2$;
3. AP avec amortissement $\lambda = 0{,}9$ et préférence réglée par **quantile** q des similarités (comme l'argument `q` d'`apcluster`) ;
4. **balayage** de q de 0 à 1 par pas de 0,05 pour mesurer la sensibilité ;

5. comparaison à K-means, K-medoids (`pam`) et au regroupement hiérarchique de Ward, à **nombre de groupes imposé par AP**, sur les mêmes données standardisées.

Les étiquettes d'espèces servent **uniquement à évaluer** les partitions (indice de Rand ajusté, ARI), jamais à les construire. Les scripts complets, `tp/Application_Iris.R` et `tp/Application_Iris.py`, s'exécutent depuis la racine du dépôt ; toutes les graines aléatoires sont fixées.

5.2. Résultats reproduits localement

Les chiffres ci-dessous sont des **résultats produits par nos scripts** (Python 3.12.3, scikit-learn 1.9.1 ; le script R est son pendant direct) — ce ne sont pas des résultats de la littérature.

Sensibilité à la préférence. Sur Iris standardisé, le nombre de clusters croît continûment avec le quantile de la préférence : 3 groupes au minimum des similarités ($q = 0$), 9 groupes à la médiane ($q = 0{,}5$), 12 groupes à $q = 0{,}6$, jusqu'à 149 groupes à $q = 1$. L'accord avec les espèces botaniques (ARI) est le plus élevé au minimum des similarités (ARI = 0,631 pour 3 groupes) et décroît régulièrement quand la préférence augmente (ARI 0,35 à la médiane, proche de 0 au-delà de $q = 0{,}9$). Le tableau ci-dessous, extrait de la sortie de `tp/Application_Iris.py`, résume ce balayage.

TABLE 1 – Balayage de la préférence sur Iris standardisé (scikit-learn 1.9.1, graine 2026). Le nombre de clusters croît avec q ; l'accord avec les espèces décroît.

quantile q	0	0,05	0,30	0,50	0,60	0,70	0,80	0,90	1
nb de clusters	3	5	6	9	12	16	19	33	149
ARI vs espèces	0,631	0,443	0,392	0,354	0,287	0,269	0,238	0,121	0,000

La lecture du tableau est immédiate : aucune valeur de la préférence n'est « meilleure » en soi, mais seule une préférence proche du minimum retrouve ici les trois espèces. La courbe complète (21 valeurs de q) est produite par les deux scripts du TP.

Réglage par défaut. La préférence médiane, choix par défaut d'`apcluster` comme de scikit-learn, produit ici **9 clusters** — trois fois plus que les 3 espèces réelles — avec un ARI de 0,354. Le réglage par défaut ne « retrouve » donc pas la structure botanique : il la **fragmente**. Cette observation rejoint les mises en garde de la littérature : la médiane est un point de départ, pas une garantie (SINHA et JANA 2025).

Comparaison à nombre de groupes égal ($k = 9$). À nombre de groupes imposé par AP, K-means atteint un ARI de 0,322 contre 0,354 pour AP, avec une somme des distances au carré aux représentants légèrement plus faible (54,6 contre 60,1) : chaque méthode optimise un critère différent. La version R du TP ajoute `pam` (K-medoids) et le regroupement de Ward dans la même configuration.

Amortissement. À préférence médiane fixée, l'amortissement ne change pas la partition obtenue (9 clusters pour $\lambda \in \{0{,}5; 0{,}7; 0{,}9\}$) mais allonge la convergence : 109 itérations à $\lambda = 0{,}5$, 113 à 0,7, 210 à 0,9. L'amortissement joue donc ici son rôle attendu : stabiliser les messages, sans modifier le résultat final sur ce jeu simple.

De R à Python. Les deux implémentations se ressemblent fortement, ce qui rend la comparaison d'usage instructive. En R, `apcluster()` prend la matrice de similarités en entrée et règle

la préférence par quantile (`q`), ce qui rend le balayage immédiat ; l'objet retourné expose directement clusters, exemplaires et nombre d'itérations. En Python, `AffinityPropagation` part des données et calcule lui-même la similarité ; la préférence est une valeur absolue (`preference`), qu'il faut donc convertir soi-même en quantile pour reproduire le réglage d'`apcluster`. À fonctionnalités équivalentes, l'interface d'`apcluster` est plus proche de la présentation théorique (matrice S, quantiles), tandis que scikit-learn s'intègre plus naturellement dans une chaîne de traitement Python (même API `fit`/`fit_predict` que les autres algorithmes). Sur ce TP, les deux outils produisent des partitions qualitativement identiques à réglages alignés.

5.3. Comparaison qualitative avec les méthodes de référence

Face à **K-means**, AP s'affranchit du choix préalable de k et choisit des représentants réels, mais il est plus coûteux et son nombre de groupes dépend de la préférence. Face aux **K-medoids**, qui sélectionnent eux aussi des observations réelles, AP évite de fixer k, au prix d'un réglage continu parfois délicat. Face à la **classification hiérarchique**, qui fournit toute une échelle de regroupements, AP délivre directement une partition sans découpage de dendrogramme. En contrepartie, sa mémoire en $O(n^2)$ et sa sensibilité à la préférence et à l'amortissement limitent son usage aux jeux de données de taille modeste et exigent une étude de sensibilité systématique.

5.4. Pourquoi le nombre de groupes varie-t-il ?

Le mécanisme peut se comprendre sans mathématiques lourdes. Chaque observation se demande : « suis-je assez singulière pour me représenter moi-même, ou une voisine me représente-t-elle mieux ? ». La **préférence** est le seuil d'exigence de cette question : réglée bas, une observation accepte d'être représentée par une voisine même lointaine, et peu d'exemplaires suffisent ; réglée haut, presque chaque observation s'estime irremplaçable, et les exemplaires se multiplient. Les **similarités** fixent, elles, le paysage dans lequel ce marchandage a lieu : si les données forment des groupes denses et bien séparés, un large éventail de préférences donne le même découpage ; si la structure est diffuse, le nombre de groupes glisse continûment avec la préférence — c'est ce que montre notre balayage sur Iris, où seule la préférence minimale approche les trois espèces. Le nombre de clusters d'AP n'est donc pas une propriété « découverte » des données, mais le résultat conjoint de la structure des similarités et d'un paramètre que l'analyste doit assumer et documenter.

6. Ce que dit la bibliographie sur Affinity Propagation

L'article fondateur de Frey et Dueck (2007), publié dans *Science*, introduit AP et l'illustre sur des visages, des phrases et des villes ; il recommande la préférence médiane comme point de départ. Un commentaire critique de Brusco et Köhn (2008), dans la même revue, montre qu'une heuristique classique du problème p-médiane atteint souvent une erreur plus faible qu'AP en un temps comparable : l'avantage d'AP n'est donc pas acquis et doit être évalué sur chaque cas. K. Wang et al. (2007) constatent que la préférence médiane ne conduit pas, dans de nombreux cas, à une solution satisfaisante et proposent un balayage adaptatif ; Sinha et Jana (2025) décrivent encore en 2025 le choix de la préférence comme l'un des principaux défis d'AP et mobilisent des méta-heuristiques pour le régler. Côté applications, Leone, Sumedha et Weigt (2007) appliquent une version à contraintes souples à des données d'expression génétique, Y. Wang et al. (2024) sélectionnent des conditions expérimentales en combustion par une variante à sensibilité globale,

et Periti et al. (2025) adaptent AP au suivi incrémental de corpus diachroniques. Côté implémentations, le package R `apcluster` (Bodenhofer, Kothmeier et Hochreiter 2011) est la référence en R, et scikit-learn (Pedregosa et al. 2011) fournit la classe `AffinityPropagation` utilisée dans notre TP ; le carnet de Sarkar (2019) en propose une introduction pédagogique en Python.

7. Discussion critique

Plusieurs limites doivent encadrer la portée de ces résultats. D'abord, Iris est un jeu **petit et facile** : nos chiffres valident le pipeline et illustrent le comportement d'AP, mais n'autorisent aucune conclusion sur la performance générale de la méthode. Ensuite, l'ARI mesure l'accord avec des classes botaniques connues, non une « vérité » du clustering : un ARI modéré à la préférence médiane ne signifie pas qu'AP échoue, mais qu'il découpe autrement que la taxonomie — ce qui est précisément le phénomène de fragmentation que nous documentons. Enfin, notre comparaison à k imposé par AP est équitable pour mesurer la qualité de partition, mais elle ne tranche pas la question du **choix** de k, que K-means et K-medoids délèguent à l'analyste et qu'AP délègue à la préférence. La critique de Brusco et Köhn (2008) — une heuristique du p-médiane peut atteindre une erreur plus faible qu'AP — rappelle que l'intérêt d'AP tient moins à une supériorité numérique qu'à son cadre : pas de k à fixer, des représentants réels, une similarité libre. Nos prolongements prioritaires seraient l'application au jeu Wine (13 variables, où la standardisation compte davantage) et une étude de stabilité par sous-échantillonnage, propositions que nous marquons comme hypothèses du projet plutôt que comme résultats.

8. Conclusion

Affinity Propagation occupe une place singulière dans la panoplie du clustering : plutôt que de fixer un nombre de groupes, il fait émerger des **exemplaires réels** d'un marchandage itératif entre observations, réglé par la préférence et amorti pour converger. Notre application à Iris illustre à la fois sa simplicité d'usage — quelques lignes en R comme en Python — et sa principale exigence : le nombre de groupes obtenu dépend fortement de la préférence, et le réglage par défaut peut fragmenter une structure connue. AP est donc un outil précieux lorsque l'on cherche des représentants concrets sans hypothèse sur k, à condition d'en balayer les réglages, de le confronter à K-means, K-medoids ou à la classification hiérarchique, et de rapporter paramètres, graines et versions. Les prolongements possibles incluent l'application au jeu Wine, l'étude de stabilité par sous-échantillonnage et les variantes récentes de la littérature.

9. Annexe : extraits de code

Le TP complet est fourni dans `tp/`. On en extrait ici le cœur minimal, identique dans son principe en R et en Python.

Version R (`tp/Application_Iris.R`) :

```
library(apcluster)
X <- scale(as.matrix(iris[, 1:4]))     # z-score
S <- negDistMat(X, r = 2)              # s(i,k) = -||xi - xk||^2
res <- apcluster(S, q = 0.5, lam = 0.9, seed = 2026)
length(res@clusters)                   # nombre de groupes trouvés
res@exemplars                          # observations choisies comme exemplaires
```

Version Python (`tp/Application_Iris.py`) :

```
from sklearn.cluster import AffinityPropagation
from sklearn.preprocessing import StandardScaler

X_std = StandardScaler().fit_transform(iris.data)
ap = AffinityPropagation(damping=0.9, random_state=2026)
labels = ap.fit_predict(X_std)
exemplaires = ap.cluster_centers_indices_   # observations réelles
```

Bibliographie

Bodenhofer, Ulrich, Andreas Kothmeier et Sepp Hochreiter (2011). « APCluster : an R package for affinity propagation clustering ». In : *Bioinformatics* 27.17, p. 2463-2464. doi : 10.1093/bioinformatics/btr406.

Brusco, Michael J. et Hans-Friedrich Köhn (2008). « Comment on "Clustering by Passing Messages Between Data Points" ». In : *Science* 319.5864, p. 726-726. doi : 10.1126/science.1150921.

Frey, Brendan J. et Delbert Dueck (2007). « Clustering by Passing Messages Between Data Points ». In : *Science* 315.5814, p. 972-976. doi : 10.1126/science.1136800.

Leone, Michele, Sumedha et Martin Weigt (2007). « Clustering by soft-constraint affinity propagation : applications to gene-expression data ». In : *Bioinformatics* 23.20, p. 2708-2715. doi : 10.1093/bioinformatics/btm414.

Pedregosa, Fabian et al. (2011). « Scikit-learn : Machine Learning in Python ». In : *Journal of Machine Learning Research* 12, p. 2825-2830. url : https://jmlr.org/papers/v12/pedregosa11a.html.

Periti, Francesco et al. (2025). « Incremental Affinity Propagation Based on Cluster Consolidation and Stratification ». In : *Neural Processing Letters* 57, p. 44. doi : 10.1007/s11063-025-11752-y.

Sarkar, Tirthajyoti (2019). *Affinity Propagation Clustering Technique (carnet de démonstration)*. https://github.com/tirthajyoti/Machine-Learning-with-Python/blob/master/Clustering-Dimensionality-Reduction/Affinity_Propagation.ipynb. Dépôt GitHub *Machine-Learning-with-Python*, consulté le 2 novembre 2026.

Sinha, Abhinav et Prasanta K. Jana (2025). « Improved affinity propagation clustering algorithms : a PSO-based approach ». In : *Knowledge and Information Systems* 67, p. 1011-1041. doi : 10.1007/s10115-024-02260-x.

Wang, Kaijun et al. (2007). « Adaptive Affinity Propagation Clustering ». In : *Acta Automatica Sinica* 33.12, p. 1242-1246. doi : 10.1360/aas-007-1242.

Wang, Y. et al. (2024). « Clustering algorithm for experimental datasets using global sensitivity-based affinity propagation (GSAP) ». In : *Com-*

bustion and Flame 259, p. 113121. DOI : 10.1016/j.combustflame.2023.113121.